

# Privacy-Preserving Voting System Using Local Differential Privacy

Phan Anh Le, Philip Pearce-Pearson, Kushal Sigdel

December 2025

## 1 Introduction

Online voting is a task that fundamentally requires privacy: individuals should be able to express their choices and opinions without fear that their preferences can be revealed or inferred. However, most online polling systems store raw votes directly in a database. Even if names are not stored, votes can often be linked back to individuals through metadata or correlation attacks.

This project addresses that problem by designing a full-stack polling and voting system that integrates privacy at the moment of data collection. Instead of storing the true vote, the backend applies a local differential privacy (LDP) mechanism. As a result, the server never learns the true vote and only receives a randomized version, while still enabling statistically valid estimation of aggregate poll results.

The goal of the project was to apply the concepts of the Data Privacy course into a practical project by building a functional online voting platform and incorporating practical differential privacy techniques directly into the system.

## 2 Motivation and Novelty

Polling platforms are widely used in social media, education, marketing, and research. Despite handling sensitive preferences, most of them forgot the significance of privacy for their databases. This creates severe privacy risks if the databases are leaked, compromised, or misused.

The novelty of this project lies in integrating **local differential privacy** into a real application. Unlike central differential privacy, where the server collects raw data and adds noise during query time, LDP protects each user before the data leaves their device. This means:

- the server never sees the true vote,

- the database contains only noisy votes,
- even with full database access, an attacker cannot determine individual preferences.

## 3 System Structure

The system consists of a FastAPI backend, a SQLite database, and two frontend interfaces.

### 3.1 Backend (FastAPI)

The backend provides REST endpoints for poll creation, option addition, vote submission, poll closing, and result retrieval. The privacy logic resides in:

- `/vote` (applies randomized response)
- `/results` (debiases noisy vote counts).

### 3.2 Frontend Interfaces

Two HTML/JavaScript interfaces were built:

- A **creator dashboard** for creating and managing polls.
- A **voting page** for loading polls and submitting votes.

Each user receives a persistent anonymous voter token stored locally to enforce one-vote-per-poll while preserving anonymity.

### 3.3 Database Schema

The SQLite database includes four tables:

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <b>polls</b>        | Stores question IDs, questions, created time, and poll status. |
| <b>options</b>      | Stores option IDs, poll IDs, and texts for each poll.          |
| <b>vote_records</b> | Prevents duplicate votes.                                      |
| <b>votes</b>        | Stores noisy, randomized votes.                                |

The **votes** table is the core of the privacy mechanism.

## 4 Privacy Mechanism: Local Differential Privacy

### 4.1 Why Local Differential Privacy?

Local DP guarantees privacy for each individual, even if:

- the server is malicious,
- the database leaks,
- data is analyzed adversarially.

In a voting context, this ensures that no party can determine how a specific user voted.

### 4.2 Local DP Mechanism

Suppose a user selects option  $o$  from  $k$  possible options. Before storing the vote, the system perturbs it as follows:

- With probability  $(1 - p)$ , record the true option  $o$ .
- With probability  $p$ , record a random other option.

The parameter  $p$  is derived from the privacy budget  $\epsilon$ :

$$p = \frac{e^\epsilon}{e^\epsilon + (k - 1)}, q = \frac{1 - p}{k - 1},$$

This mechanism guarantees  $\epsilon$ -local differential privacy.

### 4.3 Effect on Stored Data

Traditional systems store:

`option_id = 3` (meaning the user selected option 3)

Under LDP, the database may instead store:

`option_id = 1`

even if the true selection was option 3. Thus, a single stored vote reveals nothing certain about the user's choice.

## 4.4 Debiasing Aggregate Results

Let:

- $N$  = total number of voters,
- $k$  = number of options,
- $r_i$  = reported (noisy) response for user  $i$ ,
- $p_i$  = probability that user  $i$  reports the true option under their privacy mechanism,
- $q_i$  = probability that user  $i$  reports option  $j$  when their true option is *not*  $j$ .

The unbiased estimator of the true count  $n_j$  is:

$$\hat{T}_j = \sum_{i=1}^N \frac{1(r_i = j) - q_i}{p_i - q_i}.$$

This expression works by removing the expected noise from each user's randomized report. The numerator  $1(r_i = j) - q_i$  subtracts the chance of a false randomized response, while the denominator  $p_i - q_i$  rescales the contribution to correct for the user's personalized randomization probabilities. Summing over all users yields an unbiased estimate of the true number of votes for option  $j$  without ever storing any individual's actual vote..

## 5 Before and After Privacy Comparison

### Before (No Privacy)

- Exact votes were stored.
- Anyone with DB access could see preferences.
- Individual votes were fully linkable.

### After (With LDP)

- Only randomized votes are stored.
- Individual preferences cannot be inferred with confidence.
- Aggregate results are reconstructed statistically.
- Strong privacy holds even if the server is compromised.

This represents a major improvement in user protection.

## 6 Results and Analysis: An Experiment

To evaluate the accuracy and behavior of our privacy-preserving voting mechanism, we ran a sequence of experiments comparing **true vote counts** with the **debiased estimates** returned by the backend. Synthetic users voted under a weighted randomized response mechanism, and we varied the number of users

$$N \in \{50, 100, 200, 500, 1000\},$$

running five trials for each setting and averaging the results.

| Professor       | True Avg (N=50) | Debiased Avg (N=50) | True Avg (N=100) | Debiased Avg (N=100) |
|-----------------|-----------------|---------------------|------------------|----------------------|
| Prof. Kretchmar | 8.40            | 17.00               | 19.80            | 9.88                 |
| Prof. Law       | 4.00            | 3.49                | 5.60             | 23.55                |
| Prof. Lall      | 6.60            | 3.22                | 15.00            | 40.96                |
| Prof. Truex     | 31.00           | 57.28               | 59.60            | 53.82                |

| Professor       | True Avg (N=200) | Debiased Avg (N=200) | True Avg (N=500) | Debiased Avg (N=500) |
|-----------------|------------------|----------------------|------------------|----------------------|
| Prof. Kretchmar | 42.60            | 25.73                | 98.00            | 115.40               |
| Prof. Law       | 8.60             | 17.70                | 28.60            | 70.10                |
| Prof. Lall      | 30.20            | 92.45                | 73.60            | 16.77                |
| Prof. Truex     | 118.60           | 78.21                | 299.80           | 303.99               |

| Professor       | True Avg (N=1000) | Debiased Avg (N=1000) |
|-----------------|-------------------|-----------------------|
| Prof. Kretchmar | 207.20            | 267.27                |
| Prof. Law       | 46.00             | 36.66                 |
| Prof. Lall      | 145.40            | 136.01                |
| Prof. Truex     | 601.40            | 584.63                |

Table 1: Comparison of true average counts and debiased average estimates across five trials for each sample size.

### 6.1 Performance at Small Sample Sizes

At  $N = 50$ , the debiased estimates deviate substantially from the true counts (Figure 1). This is clearest for the high-probability, heavily weighted option (Prof. Truex), whose debiased average increases to 57.28 despite a true mean of 31.00. Similarly, lower-probability options such as Prof. Law and Prof. Lall oscillate between overestimation and underestimation. This pattern is expected because local differential privacy (LDP) mechanisms introduce strong individual-level noise, and the debiasing estimator amplifies variance when  $N$  is small.

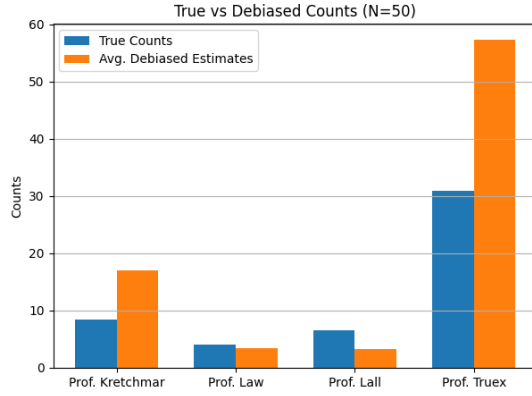


Figure 1: True vs. Debiased Counts for  $N = 50$ .

## 6.2 Moderate Sample Sizes

At  $N = 100$  and  $N = 200$ , accuracy improves but noticeable noise persists (Figures 2–3). For example:

- At  $N = 100$ , Prof. Lall’s true mean of 15.00 becomes an inflated debiased estimate of 40.96.
- At  $N = 200$ , Prof. Lall again experiences overestimation (92.45 vs. 30.20), whereas Prof. Kretchmar and Prof. Truex move closer to their true values.

These mid-range discrepancies reflect the inherent instability of debiasing for low-frequency categories, especially when weighted privacy enhances perturbation.

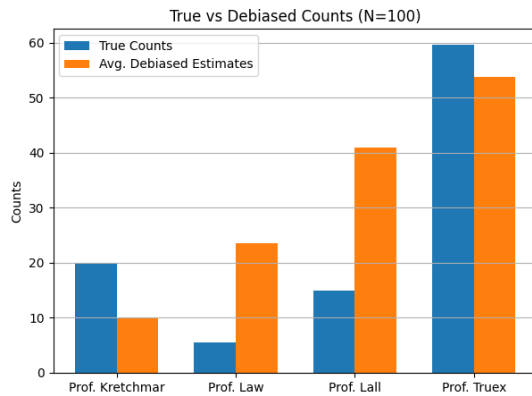


Figure 2: True vs. Debiased Counts for  $N = 100$ .

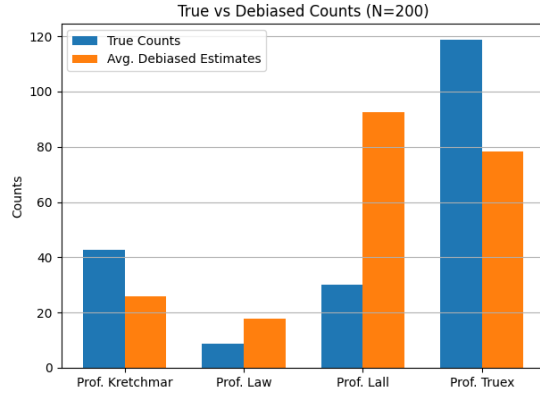


Figure 3: True vs. Debiased Counts for  $N = 200$ .

### 6.3 Convergence at Larger Sample Sizes

At  $N = 500$  and  $N = 1000$ , the debiased estimates converge much more closely toward the true counts (Figures 4–5). Notable examples include:

- Prof. Truex is estimated extremely accurately: 303.99 vs. 299.80 at  $N = 500$ , and 584.63 vs. 601.40 at  $N = 1000$ .
- Prof. Lall achieves strong convergence at  $N = 1000$  (136.01 vs. 145.40).

This trend matches theoretical expectations: as  $N$  grows, randomized response becomes asymptotically unbiased with diminishing variance.

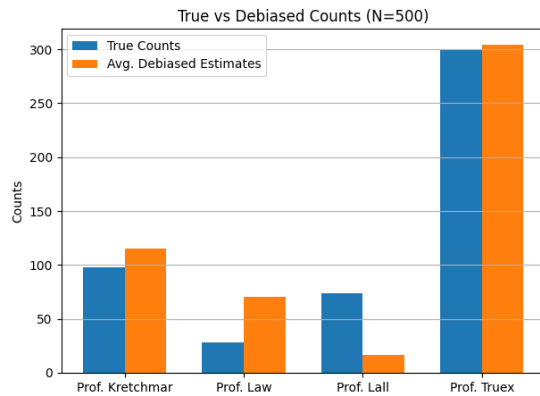


Figure 4: True vs. Debiased Counts for  $N = 500$ .

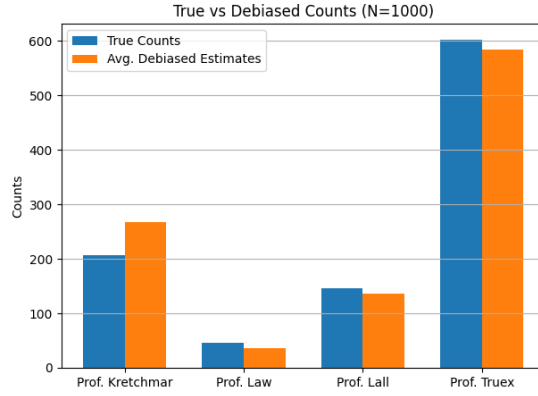


Figure 5: True vs. Debiased Counts for  $N = 1000$ .

## 6.4 Effect of Weights and Heterogeneous Privacy

Two design features shape the observed variance:

1. **Creator-specified weights.** Prof. Truex receives the highest weight (2.0), increasing noise for that option. The inflated debiased counts for smaller  $N$  align with this increased protection.
2. **User-sampled privacy budgets.** Each user draws  $\epsilon \in \{0.5, 1.0, 2.0\}$ , producing heterogeneous noise. Despite this, the debiasing procedure consistently reconstructs the distribution for  $N \geq 500$ .

## 6.5 Summary of Findings

Across all experiments, the system shows:

- High variance at small  $N$  due to strong LDP noise.
- Clear improvement at moderate  $N$ , though rare options remain unstable.
- Strong convergence for  $N \geq 500$ , demonstrating accurate aggregate recovery despite locally privatized votes.
- Robustness to both heterogeneous privacy parameters and option-specific weights.

These findings confirm that practical polling systems can achieve strong privacy while still producing reliable population-level statistics.



## 7 Future Improvements

Several enhancements could further strengthen both the privacy guarantees and the practical usability of the system:

- **Adaptive or Personalized Privacy Budgets.** While this project supports heterogeneous  $\epsilon$  values, future versions could allow users to dynamically adjust their privacy level in real time, or adopt mechanisms that automatically tune  $\epsilon$  based on user behavior, poll sensitivity, or aggregate uncertainty.
- **Enhanced Vote Integrity and Abuse Prevention.** While one-vote-per-user is enforced through anonymous tokens, stronger protections, such as rate limiting, blinded signatures, or lightweight zero-knowledge proofs, could ensure that privacy-preserving mechanisms remain robust against adversarial clients.
- **Support for Additional Privacy Models.** Beyond randomized response, the platform could support secure aggregation or homomorphic tallying to allow comparisons between local and central differential privacy models. This would offer a richer experimental environment and additional pathways to high-accuracy, privacy-preserving analytics.
- **Improved User Interface for Privacy Transparency.** Future iterations could integrate visual explanations of how  $\epsilon$  affects accuracy, provide interactive sliders for tuning privacy levels, or include user-facing summaries explaining why results may appear noisy.

## 8 Conclusion

This project demonstrates that local differential privacy can be successfully embedded into a functional, end-to-end voting platform. By applying a weighted variant of randomized response and allowing users to draw heterogeneous privacy budgets, the system ensures that no individual vote is ever stored in its true form. Instead, only privatized, noisy reports reach the backend, offering strong protection even under server compromise.

Through constructing the database, backend logic, debiasing estimator, and frontend interfaces, this project highlighted the practical realities of privacy engineering: noise interacts with system design, debiasing requires careful statistical reasoning, and accuracy is fundamentally tied to participation size. The experimental evaluation confirmed theoretical expectations that while aggregate accuracy is limited for small samples, the debiased estimates converge rapidly as the number of users increases.

The final system serves as a concrete demonstration of how differential privacy principles can be applied to tasks such as polling and preference collection. It highlights both the promise and the

challenges of building privacy-preserving applications and illustrates that meaningful analytics can be achieved without ever compromising individual confidentiality.